

ALGI

Langage de Programmation en Langue Balengou

Travaux Pratiques de Compilation - Master 1 Informatique

NGUEUDJANG DJOMO ALAIN GILDAS - 22W2183

Université de Yaoundé I
Faculté des Sciences
Département d'Informatique

11 juillet 2026

Superviseur : Dr. THOMAS MESSI NGUELE

Plan de la Présentation

- 1 Introduction
- 2 Analyse Lexicale
- 3 Analyse Syntaxique
- 4 Table des Symboles
- 5 Génération de Code
- 6 Runtime
- 7 Exemples
- 8 Compilation et Utilisation
- 9 Tests et Qualité
- 10 Conclusion

Objectifs

- Créer un langage de programmation avec des mots-clés inspirés de la langue **balengou**
- Valoriser et documenter un corpus lexical d'une langue camerounaise
- Démonstration de la faisabilité d'un langage formel construit sur une langue africaine

Problématique

- Comment concevoir un compilateur complet pour un langage de programmation ?
- Quels sont les étapes et outils nécessaires ?
- Comment intégrer une dimension culturelle dans un projet technique ?

Pourquoi ALGI ?

Dimension Technique

- Apprentissage des techniques de compilation
- Maîtrise des outils (Flex, Bison, GCC)
- Génération de code (C et Assembleur)

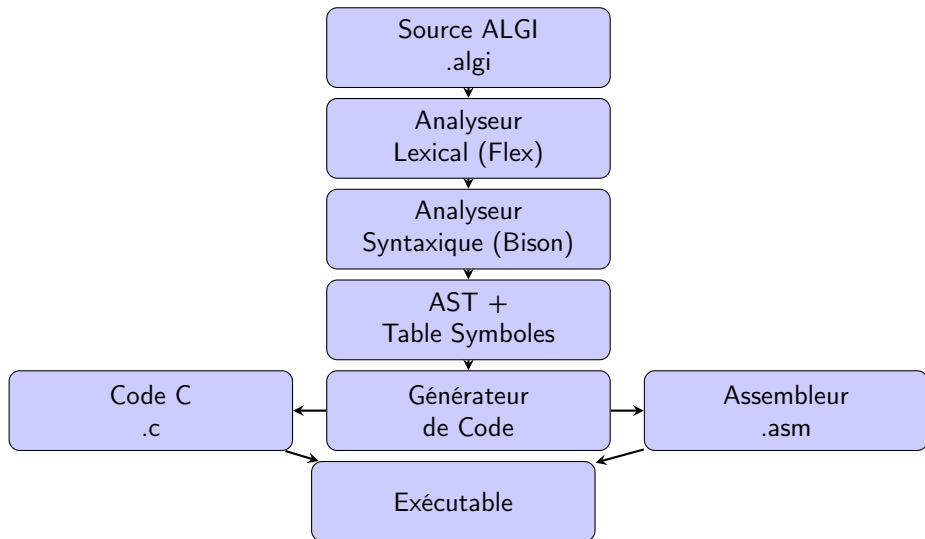
Dimension Culturelle

- Valorisation des langues locales
- Accessibilité à la programmation
- Identité et diversité culturelle

Signification

ALGI = *Alain gildas* (inspiré de la langue balengou)

Architecture Globale du Compilateur



Rôle

- Reconnaître les tokens (mots-clés, identifiants, nombres, etc.)
- Ignorer les commentaires et espaces
- Gérer les erreurs lexicales

Caractéristiques

- 34 mots-clés en bamiléké
- Expressions régulières pour les tokens
- Gestion des chaînes de caractères
- Comptage des lignes et colonnes

Exemple du Lexer

```
1 // Mots-clés en bamiléké
2 "ke"           { return KEYWORD_IF; }
3 "kommenue"    { return KEYWORD_ELSE; }
4 "w"           { return KEYWORD_WHILE; }
5 "pou"         { return KEYWORD_FOR; }
6 "mi"          { return KEYWORD_END; }
7 "he"          { return KEYWORD_LET; }
8 "fonksion"    { return KEYWORD_FUNCTION; }
9 "kwouo"       { return KEYWORD_CLASS; }
10 "feh"         { return KEYWORD_NEW; }
11
12 // Expressions régulières
13 {INTEGER}      { yylval.number = atof(yytext); return NUMBER; }
14 {FLOAT}        { yylval.number = atof(yytext); return NUMBER; }
15 {IDENTIFIER}   { yylval.identifier = strdup(yytext); return
    IDENTIFIER; }
```

Listing 1 – Extrait de algi.l

Table des Mots-Clés ALGI

ALGI	Français	ALGI	Français	ALGI	Français
ke	si	kommenu	sinon	pok	faux
w	tant que	pou	pour	n	nul
mi	fin	he	déclaration	e	et
fonsion	fonction	kwouo	classe	ou	ou
feh	nouveau	tem	retour	poke	non
peu	casser	tela	essayer	est	est
tatem	attraper	tehap	afficher	dans	dans
keksie	lire	sanwe	évaluer	comme	comme
waan	somme	tie	soustraire	bi	liste
denden	égal	nwagniehwe	trier	saa	entier
ge	typeof	ganfa	ouvrir	reyel	réel
stegnie	exit	nene	vrai	djanghom	chaîne

Rôle

- Vérifier la grammaire du langage
- Construire l'AST (Arbre Syntaxique Abstrait)
- Gérer les erreurs syntaxiques

Grammaire ALGI

- 50+ règles de production
- Gestion des opérateurs et priorité
- Support des structures de contrôle
- Support des fonctions et classes

Exemple de la Grammaire

```
1 // Structures de contrôle
2 if_statement
3     : KEYWORD_IF '(' expression ')' block KEYWORD_END {
4         $$ = create_if($3, $5, NULL);
5     }
6     | KEYWORD_IF '(' expression ')' block KEYWORD_ELSE block
7     KEYWORD_END {
8         $$ = create_if($3, $5, $7);
9     }
10    ;
11 // Boucles
12 while_loop
13     : KEYWORD_WHILE '(' expression ')' block KEYWORD_END {
14         $$ = create_while($3, $5);
15     }
16     ;
17 for_loop
18     : KEYWORD_FOR '(' IDENTIFIER OPERATOR_IN expression ')' block
19     KEYWORD_END {
20         $$ = create_for($3, $5, $7);
21     }
22     ;
```

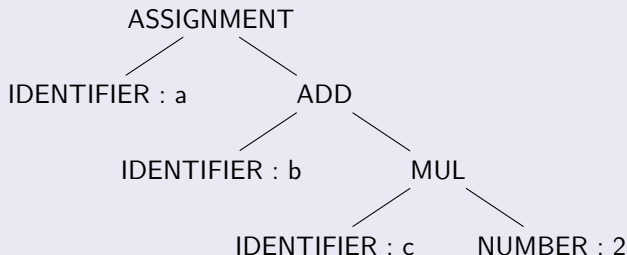
Listing 2 – Extrait de algi.y

Arbre Syntaxique Abstrait (AST)

Nœuds Principaux

- **NODE_PROGRAM** : Programme principal
- **NODE_FUNCTION** : Définition de fonction
- **NODE_IF** / **NODE_WHILE** / **NODE_FOR** : Structures
- **NODE_BINARY_OP** / **NODE_UNARY_OP** : Opérateurs
- **NODE_LIST** / **NODE_CALL** : Listes et appels

Exemple d'AST pour $a = b + c * 2$



Rôle

- Stocker les informations sur les identifiants
- Gérer les portées (scopes)
- Vérifier les déclarations
- Support des types

Informations Stockées

- Nom du symbole
- Type (variable, fonction, classe, paramètre, champ, méthode)
- Type de données (int, float, string, bool, list, object)
- Niveau de portée
- Mutabilité

Structure de la Table des Symboles

```
1 typedef struct Symbol {  
2     char* name;  
3     SymbolKind kind;  
4     DataType data_type;  
5     int scope_level;  
6     int is_mutable;  
7     struct Symbol* next;  
8 } Symbol;  
9  
10 typedef struct SymbolTable {  
11     Symbol** symbols;  
12     int count;  
13     int capacity;  
14     int scope_level;  
15     struct SymbolTable* parent;  
16 } SymbolTable;
```

Listing 3 – symbol.h

Caractéristiques

- Génération de code C portable
- Utilisation du runtime `AlgiValue*`
- Support complet des fonctionnalités ALGI
- Gestion dynamique des types

Runtime

- Types dynamiques (INT, FLOAT, STRING, BOOL, LIST, OBJECT)
- Fonctions d'opérations arithmétiques
- Gestion des exceptions (`setjmp/longjmp`)
- Fonctions natives (`print`, `input`, `sum`, `sort`, etc.)

Exemple de Code C Généré

```
1 he x = 10
2 he y = 20
3 tehap(x + y)
```

Listing 4 – Code ALGI

```
1 #include <stdio.h>
2 #include "runtime.h"
3
4 int main(void) {
5     AlgiValue* x = algi_make_int(10);
6     AlgiValue* y = algi_make_int(20);
7     algi_print(algi_add(x, y));
8     return 0;
9 }
```

Listing 5 – Code C Généré

Caractéristiques

- Génération de code x86 32 bits
- Utilisation de NASM
- Pédagogique : montre le fonctionnement bas niveau
- Support des entiers, boucles, conditions

Limitations

- Pas de support des chaînes dynamiques
- Pas de flottants
- Pas de listes/objets
- Générateur simplifié pour l'apprentissage

Exemple de Code Assembleur Généré

```
1  
2 he a = 5  
3 he b = 3  
4 tehap(a + b)
```

code asm partie 1

```
1 section .data  
2     format_int db "%d", 10, 0  
3 section .text  
4     global _start  
5 _start:  
6     push ebp  
7     mov ebp, esp  
8     sub esp, 8  
9     ; a = 5  
10    mov eax, 5  
11    mov [ebp-4], eax  
12    ; b = 3  
13    mov eax, 3
```

code asm partie 2

```
1     mov [ebp-8], eax  
2     ; a + b  
3     mov eax, [ebp-4]  
4     add eax, [ebp-8]  
5     ; affichage  
6     push eax  
7     push format_int  
8     call printf  
9     add esp, 8  
10    mov esp, ebp  
11    pop ebp  
12    ret
```

Composants Principaux

- **AlgValue** : Structure de données dynamique
- **AlgList** : Liste chaînée dynamique
- **AlgObject** : Objet avec champs
- **Gestion des exceptions** : setjmp/longjmp
- **Fonctions natives** : print, input, sum, etc.

Structure Algivalue

```
1  typedef enum {
2      TYPE_INT = 1,
3      TYPE_FLOAT = 2,
4      TYPE_STRING = 3,
5      TYPE_BOOL = 4,
6      TYPE_LIST = 5,
7      TYPE_OBJECT = 6,
8      TYPE_VOID = 7,
9      TYPE_NULL = 8,
10     TYPE_ERROR = 9
11 } Algitype;
12
13 typedef struct {
14     Algitype type;
15     union {
16         long int_value;
17         double float_value;
18         bool bool_value;
19         char* string_value;
20         void* ptr_value;
21     } value;
22 } Algivalue;
```

Listing 6 – runtime.h

```
1 // Création
2 Algivalue* algi_make_int(long value);
3 Algivalue* algi_make_string(const char* value);
4 Algivalue* algi_make_list(Algilist* list);
5
6 // Opérations
7 Algivalue* algi_add(Algivalue* a, Algivalue* b);
8 Algivalue* algi_sub(Algivalue* a, Algivalue* b);
9 Algivalue* algi_equals(Algivalue* a, Algivalue* b);
10
11 // Gestion des erreurs
12 void algi_throw(const char* error_message);
13 void algi_push_try(void);
14 int algi_setjmp_try(void);
15 char* algi_get_last_error(void);
```

Listing 7 – Exemples de fonctions

Exemple 1 : Bonjour le Monde

```
1 // Bonjour le monde en ALGI
2 tehap("Bonjour le monde!")
3 tehap("Bienvenue dans ALGI!")
4
5 he message = "ALGI est un langage bamiléké"
6 tehap(message)
7
8 // Affichage d'une variable
9 he nom = "Alain"
10 tehap("Bonjour " + nom + "!" )
```

Listing 8 – bonjour.algi

```
1 Bonjour le monde!
2 Bienvenue dans ALGI!
3 ALGI est un langage bamiléké
4 Bonjour Alain!
```

Listing 9 – Résultat

Exemple 2 : Calculatrice Simple

```
1  tahap("=== Calculatrice ALGI ===")
2
3  tahap("Entrez le premier nombre:")
4  he a = keksie()
5
6  tahap("Entrez le deuxième nombre:")
7  he b = keksie()
8
9  tahap(a + " + " + b + " = " + (a + b))
10 tahap(a + " - " + b + " = " + (a - b))
11 tahap(a + " * " + b + " = " + (a * b))
12
13 ke (b != 0)
14     tahap(a + " / " + b + " = " + (a / b))
15 kkommen
16     tahap("Division par zéro impossible!")
17 mi
```

Listing 10 – calculatrice.algi

Exemple 3 : Tri par Insertion

```
1  fonksion triinsertion(liste)
2      he n = liste
3      he i = 1
4      w (i < n)
5          he element = n[i]
6          he j = i - 1
7          w (j >= 0 e n[j] > element)
8              n[j + 1] = n[j]
9              j = j - 1
10         mi
11         n[j + 1] = element
12         i = i + 1
13     mi
14     tem n
15 mi
16
17 he nombres = bi[5, 2, 8, 1, 3]
18 tehap("Liste originale: " + nombres)
19 he tries = triinsertion(nombres)
20 tehap("Liste triée: " + tries)
```

Listing 11 – triinsertion.algi

Prérequis

- GCC (ou Clang)
- Flex
- Bison
- Make
- NASM (pour le mode assembleur)

Commande	Description
<code>make</code>	Compiler le compilateur
<code>make clean</code>	Nettoyer les fichiers
<code>make clean-all</code>	Nettoyage complet
<code>make run-c</code>	Compiler C et exécuter
<code>make run-asm</code>	Compiler ASM et exécuter
<code>make test-all</code>	Exécuter tous les tests
<code>make install</code>	Installer le compilateur
<code>make help</code>	Afficher l'aide

Types de Tests

- **Unitaires** : Chaque module testé individuellement
- **Intégration** : Interaction entre les modules
- **Fonctionnels** : Programmes ALGI complets
- **Régression** : Non-régression après modifications

Exemples de Tests

- `bonjour.algi` : Base
- `variables.algi` : Variables et types
- `calculatrice.algi` : Opérations
- `boucles.algi` : Structures de contrôle
- `fonctions.algi` : Fonctions
- `listes.algi` : Listes
- `exceptions.algi` : Gestion d'erreurs

Fonctionnalités Implémentées

- **Lexer** : 34 mots-clés en bamiléké
- **Parser** : 50+ règles de production
- **AST** : 20+ types de nœuds
- **Table des symboles** : Gestion des portées
- **Générateur C** : Code portable et complet
- **Générateur ASM** : Code x86 pédagogique
- **Runtime** : Types dynamiques et exceptions
- **Tests** : 25+ programmes de test

Défis Techniques

- Cohérence entre AST et générateur de code
- Gestion des types dynamiques en C
- Génération de code assembleur correct
- Gestion des exceptions avec setjmp/longjmp

Bugs Critiques Corrigés

- `keksie()` : Type de retour incorrect
- `est` : Comparaison de type vs valeur
- Fonctions : Paramètres perdus
- `bi[...]` : Éléments de liste perdus

Améliorations Futures

- Optimisation du code généré
- Support des types de données avancés
- Compilation JIT (Just-In-Time)
- Environnement de développement intégré (IDE)

Extensions Possibles

- Bibliothèque standard ALGI
- Support des structures de données avancées
- Programmation orientée objet complète
- Interopérabilité avec C et Python

Merci !

Questions ?

Auteur : NGUEUDJANG DJOMO ALAIN GILDAS

Email : gildas.ngueudjang@facsciences-uy1.cm

GitHub : github.com/ALAIN-NG/ALGI

*Ce projet a été réalisé dans le cadre du Master 1 en Informatique
à l'Université de Yaoundé I, Cameroun*



Flex :

<https://github.com/westes/flex>



Bison :

<https://www.gnu.org/software/bison/>



Aho, A.V., Lam, M.S., Sethi, R., Ullman, J.D. (2007) :
Compilers : Principles, Techniques, and Tools. Addison-Wesley.



NASM :

<https://www.nasm.us/>



Projet ALGI :

<https://github.com/ALAIN-NG/ALGI>